Bayesian Logistic Regression

Fedor Bentsa

2026-04-13

Load and prepare data

```
movies <- read.csv("tmdb_5000_movies.csv", stringsAsFactors = FALSE)

movies <- movies %>%
  select(
    title,
    budget,
    revenue,
    runtime,
    popularity,
    genres,
    release_date,
    original_language,
    vote_average,
    vote_count
  )

movies <- movies %>%
  filter(
    !is.na(budget),
    !is.na(revenue),
    !is.na(runtime),
    !is.na(popularity),
    !is.na(genres),
    !is.na(release_date),
    budget > 0,
    revenue >= 0,
    runtime > 0,
    popularity > 0
  ) %>%
  mutate(
    success = ifelse(revenue > budget, 1, 0),
    release_date = ymd(release_date),
    release_month = month(release_date),
    is_english = ifelse(original_language == "en", 1, 0),
    log_budget = log(budget),
    log_popularity = log(popularity + 1)
  ) %>%
  filter(!is.na(release_month))
```

```r
dim(movies)
```

```
## [1] 3762   15
```

```r
head(movies[, c("title", "budget", "revenue", "success")])
```

```
##                                      title    budget    revenue success
## 1                                   Avatar 237000000 2787965087       1
## 2 Pirates of the Caribbean: At World's End 300000000  961000000       1
## 3                                  Spectre 245000000  880674609       1
## 4                    The Dark Knight Rises 250000000 1084939099       1
## 5                              John Carter 260000000  284139100       1
## 6                             Spider-Man 3 258000000  890871626       1
```

Extract genre information

```r
extract_genres <- function(genre_string) {
  out <- tryCatch({
    if (is.na(genre_string) || genre_string == "") return(character(0))
    parsed <- fromJSON(genre_string)
    if (!is.data.frame(parsed)) return(character(0))
    if (!("name" %in% names(parsed))) return(character(0))
    as.character(parsed$name)
  }, error = function(e) character(0))

  return(out)
}

genre_list <- lapply(movies$genres, extract_genres)
all_genres <- unlist(genre_list)
top_genres <- names(sort(table(all_genres), decreasing = TRUE))[1:5]

top_genres
```

```
## [1] "Drama"     "Comedy"    "Thriller"  "Action"    "Adventure"
```

Create genre dummy variables

```r
for (g in top_genres) {
  clean_name <- paste0("genre_", gsub("[^A-Za-z0-9]", "_", tolower(g)))
  movies[[clean_name]] <- sapply(genre_list, function(x) as.integer(g %in% x))
}

model_data <- movies %>%
  select(
    success,
    log_budget,
```

```r
    log_popularity,
    runtime,
    release_month,
    is_english,
    starts_with("genre_")
  ) %>%
  drop_na() %>%
  mutate(
    log_budget = as.numeric(scale(log_budget)),
    log_popularity = as.numeric(scale(log_popularity)),
    runtime = as.numeric(scale(runtime)),
    release_month = as.numeric(scale(release_month))
  )

head(model_data)
```

```
##   success log_budget log_popularity  runtime release_month is_english
## 1       1   1.377089      2.1509199 2.553604    1.49352245          1
## 2       1   1.497394      2.0744329 2.893324   -0.57041648          1
## 3       1   1.394032      1.8226026 1.874163    0.90382561          1
## 4       1   1.404343      1.8663111 2.699198    0.01928036          1
## 5       1   1.424360      0.9584574 1.097660   -1.16011331          1
## 6       1   1.420419      1.8952130 1.437380   -0.57041648          1
##   genre_drama genre_comedy genre_thriller genre_action genre_adventure
## 1           0            0              0            1               1
## 2           0            0              0            1               1
## 3           0            0              0            1               1
## 4           1            0              1            1               0
## 5           0            0              0            1               1
## 6           0            0              0            1               1
```

Split into training and testing sets

```r
set.seed(4420)

n <- nrow(model_data)
train_index <- sample(1:n, size = floor(0.8 * n))

train_data <- model_data[train_index, ]
test_data <- model_data[-train_index, ]

dim(train_data)
```

```
## [1] 3009   11
```

```r
dim(test_data)
```

```
## [1] 753  11
```

Fit Bayesian logistic regression

```r
bayes_model <- stan_glm(
  success ~ .,
  data = train_data,
  family = binomial(link = "logit"),
  prior = normal(0, 2.5, autoscale = TRUE),
  prior_intercept = normal(0, 5, autoscale = TRUE),
  chains = 4,
  iter = 2000,
  seed = 4420,
  refresh = 0
)

summary(bayes_model)
```

```
##
## Model Info:
##  function:     stan_glm
##  family:       binomial [logit]
##  formula:      success ~ .
##  algorithm:    sampling
##  sample:       4000 (posterior sample size)
##  priors:       see help('prior_summary')
##  observations: 3009
##  predictors:   11
##
## Estimates:
##                   mean   sd   10%   50%   90%
## (Intercept)      0.9    0.2  0.6   0.9   1.1
## log_budget      -0.3    0.1 -0.3  -0.3  -0.2
## log_popularity   1.5    0.1  1.5   1.5   1.6
## runtime          0.2    0.1  0.2   0.2   0.3
## release_month    0.0    0.0 -0.1   0.0   0.0
## is_english       0.3    0.2  0.0   0.3   0.6
## genre_drama     -0.4    0.1 -0.5  -0.4  -0.3
## genre_comedy     0.1    0.1 -0.1   0.1   0.2
## genre_thriller  -0.3    0.1 -0.5  -0.3  -0.2
## genre_action    -0.1    0.1 -0.2  -0.1   0.1
## genre_adventure -0.1    0.1 -0.2  -0.1   0.1
##
## Fit Diagnostics:
##            mean   sd   10%   50%   90%
## mean_PPD 0.6    0.0  0.6   0.6   0.7
##
## The mean_ppd is the sample average posterior predictive distribution of the outcome variable (for de
##
## MCMC diagnostics
##                 mcse Rhat n_eff
## (Intercept)     0.0  1.0  3731
## log_budget      0.0  1.0  3901
## log_popularity  0.0  1.0  3895
## runtime         0.0  1.0  3878
```

```
## release_month   0.0  1.0  4728
## is_english      0.0  1.0  3802
## genre_drama     0.0  1.0  4026
## genre_comedy    0.0  1.0  3945
## genre_thriller  0.0  1.0  3556
## genre_action    0.0  1.0  3795
## genre_adventure 0.0  1.0  3930
## mean_PPD        0.0  1.0  4144
## log-posterior   0.1  1.0  1772
##
## For each parameter, mcse is Monte Carlo standard error, n_eff is a crude measure of effective sample
```

Posterior intervals

```r
posterior_interval(bayes_model, prob = 0.95)
```

```
##                       2.5%       97.5%
## (Intercept)      0.4121477  1.30330074
## log_budget      -0.3739405 -0.16061091
## log_popularity   1.4136034  1.67433453
## runtime          0.1145850  0.34699869
## release_month   -0.1101601  0.06918975
## is_english      -0.1123709  0.70165889
## genre_drama     -0.6018749 -0.20116561
## genre_comedy    -0.1555730  0.28552392
## genre_thriller  -0.5374376 -0.09962936
## genre_action    -0.3249583  0.14781528
## genre_adventure -0.3292377  0.20905568
```

Test set predictions

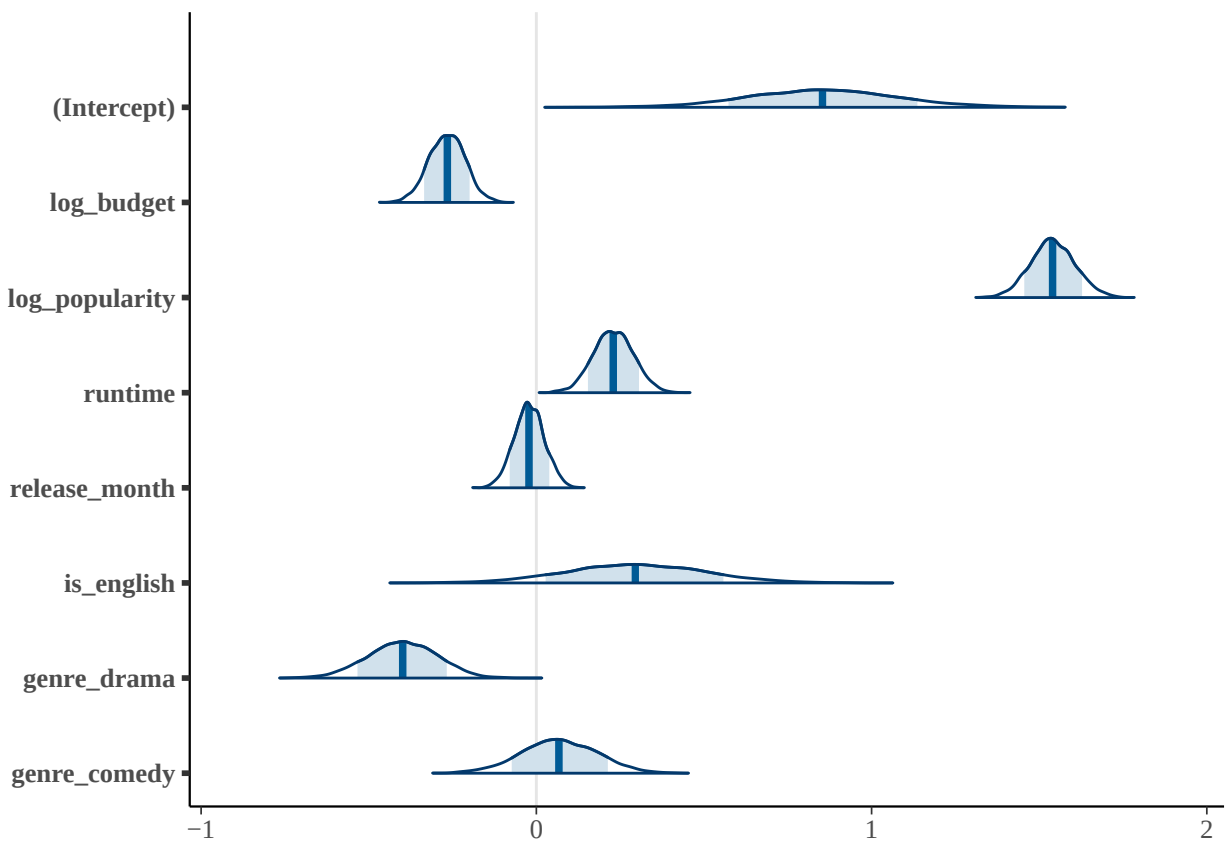
```r
posterior_probs <- posterior_linpred(
  bayes_model,
  newdata = test_data,
  transform = TRUE
)

mean_probs <- colMeans(posterior_probs)
pred_class <- ifelse(mean_probs >= 0.5, 1, 0)

accuracy <- mean(pred_class == test_data$success)
cat("Test Accuracy:", round(accuracy, 4), "\n")
```

```
## Test Accuracy: 0.7543
```

```r
conf_mat <- table(
  Predicted = pred_class,
  Actual = test_data$success
)
```

```r
conf_mat
```

```
##           Actual
## Predicted   0   1
##         0 137  58
##         1 127 431
```

Posterior plot

```r
posterior <- as.matrix(bayes_model)

mcmc_areas(
  posterior,
  pars = colnames(posterior)[1:min(8, ncol(posterior))],
  prob = 0.8
)
```